River Learning: 역전파 없는 순방향 경로 카빙

신연성
whitepep@gmail.com
2026

Abstract

본 논문은 단일 원리에 기반한 순방향 학습 알고리즘인 *River Learning*을 제시한다: 입구(입력)와 출구(출력)가 주어지면 흐름이 경로를 찾는다. 기울기도, 역방향 패스도, 전역 목적함수도 필요 없다 — 정보가 어디로 들어오고 어디에 도착해야 하는지만 알면 된다. 우리는 **Natural Optimum** 개념을 도입한다: 최적 해는 계산되거나 설계되는 것이 아니라 흐름에 의해 깎여 나온다. 강이 바다까지의 최단 경로를 계산하지 않고 단지 흘러서 뚫고 나가듯이, Natural Optimum 네트워크는 입구, 출구, 그리고 흐름이 일을 하도록 기다리는 인내만 있으면 된다. 결정적으로, **추론과 학습은 동일한 순방향 패스 내에서 발생한다**: 별도의 학습 단계도, 저장된 계산 그래프도, 학습과 배포 사이의 모드 전환도 없다. 네트워크는 처리하는 모든 입력에서 개선된다. 본 논문에서는 AND, OR, NOR, XOR을 100% 정확도로 학습하며, 정의된 입구와 출구를 가진 모든 유한 유향 네트워크 문제가 원리적으로 이 메커니즘에 의해 풀 수 있다고 추측한다.

1 서론

현대 딥러닝은 거의 전적으로 역전파[6]에 의존한다. 역전파는 미분 가능한 손실 함수, 저장된 순방향 계산 그래프, 그리고 역방향으로 오류 신호를 레이어별로 전파하는 역방향 패스를 필요로 한다. 이러한 역방향 계산 의존성은 실용적 제약(메모리, 비지역적 가중치 갱신)일 뿐 아니라 생물학적으로도 비개연적이다[1, 5].

헤비안 학습[2], 순방향-순방향[3], 타겟 전파[4] 등 여러 대안이 제안되었으나, 이들도 보조 목적함수나 전역 감독에서 파생된 레이어별 목표를 필요로 한다.

우리는 학습 문제를 환원 불가능한 최소 단위로 축소한다:

> 어디로 들어오는지 안다. 어디에 도착해야 하는지 안다. 흐르게 한다. 최적화는 계산되는 것이 아니라 — 발견되는 것이다.

이것이 우리가 **Natural Optimum**이라 부르는 것이다: 최적 경로는 설계되지도, 기울기 하강으로 찾아지지도, 탐색되지도 않는다 — 저항 매질을 통한 흐름의 물리학에서 출현한다. 손실 표면도, 기울기도, 역방향 패스도 없다. 오직 입구, 출구, 그리고 흐름 아래에서 침식되는 매질뿐이다.

River Learning은 이 원리를 형식화한다. 본 논문의 기여는 다음과 같다:

- 역방향 패스와 전역 목적함수가 없는 순방향 학습 알고리즘.
- **통합된 추론과 학습**: 예측을 생성하는 동일한 순방향 패스가 저항 갱신도 수행한다. 학습 단계도, 추론 단계도 없다 — 오직 둘 다 수행하는 단일 연산이 있을 뿐이다.
- 부울 함수(AND, OR, NOR, XOR, Parity)에서 100% 정확도의 실증.
- 정의된 입구와 출구를 가진 모든 유한 유향 네트워크는 이 메커니즘으로 학습 가능하다는 추측과 형식적 증명을 향한 스케치.

이름과 직관은 자연 지능에서 가져온다: 강, 혈관 네트워크, 점균류 등 물리 시스템이 전역 목적이나 역방향 신호 없이 국소적 침식만으로 최적 수송 경로를 발견하는 능력[7]. 강은 지형의 기울기를 계산하지 않는다. 수원지에서 시작해 바다에서 끝나며, 협곡은 저절로 형성된다.

우리는 이 프레임워크가 일반화된다고 추측한다: 정의된 입구와 출구를 가진 유향 네트워크로 표현 가능한 모든 문제는 원리적으로 흐름에 의해 학습 가능하다. 부울 함수가 첫 번째 테스트 케이스다. 라우팅, 스케줄링, 그래프 추론이 자연스러운 확장이다. 사용되는 유일한 피드백은 스칼라 이진 신호 — 출력에서 순방향 패스 후 얻을 수 있는 맞음 또는 틀림이다. 기울기는 계산되지 않는다. 역방향 패스는 없다. 네트워크는 잘못된 예측 시 활성화된 경로의 저항을 줄여서 자기 자신을 수정하며, 해당 경로가 미래에 다르게 발화할 가능성을 높인다.

비유하면 협곡을 깎는 강이다: 물은 지형의 기울기를 계산하지 않는다; 단지 흐를 뿐이며, 가장 많이 흐르는 경로가 가장 깊이 깎인다.

자연은 이것을 처음부터 해왔다. 우리가 메커니즘을 발명한 것이 아니라 — 이름을 붙인 것이다.

결과는 비유로부터 직접 따라 나온다:

- 경로는 앞으로 깎인다 — 결코 뒤로 가지 않는다. (순방향)
- 흐름은 이동하면서 침식한다 — 추론과 학습은 하나의 행위다. (위상 분리 없음)
- 힘이 필요 없다; 협곡은 스스로 형성된다. (*Natural Optimum*)
- 동일한 지형은 항상 동일한 협곡을 깎는다. (위상적 수렴)

2 방법

2.1 네트워크 구조

네트워크는 세 개의 저항 행렬로 연결된 두 개의 은닉층을 가진다:

$$\text{입력}(n_{\text{in}}) \xrightarrow{D_{12}} \text{H1}(n_{h1}) \xrightarrow{D_{23}} \text{H2}(n_{h2}) \xrightarrow{D_{34}} \text{출력}(2)$$

각 연결층은 저항 행렬 $D \in \mathbb{R}_{>0}^{m \times n}$ 로 매개변수화된다. 모든 저항은 $d_0 = 100$ 에 작은 균일 잡음을 더하여 초기화된다. 각 은닉층은 역치 벡터 $\boldsymbol{\theta} \in \mathbb{R}^n$ 도 가진다. 각 입력 x_i 는 두 개의 상보 채널 $(x_i,\, 1 - x_i)$ 로 라우팅된다: 하나가 신호를 보내지 않을 때 다른 하나가 보낸다. 이는 값에 관계없이 모든 입력에서 기본 흐름이 항상 존재하도록 보장한다. 유효 입력 차원은 $2n_{\text{in}}$ 이다.

2.2 순방향 패스

저항 행렬 D 로부터 전도도를 다음과 같이 계산한다:

$$C_{ij} = \min\left(\frac{1}{\max(d_{ij},\, \epsilon)},\ \frac{d_0}{\max(d_{ij},\, 1)}\right)$$

$\epsilon \ll 1$ 은 0으로 나누는 것을 방지한다. C_{ij} 는 저항이 작을 때(깎인 경로) 크고, 저항이 클 때(차단된 경로) 작다; $d_0/\max(d_{ij}, 1)$ 는 $d_{ij} \leq 1$ 일 때 전도도를 d_0 으로 제한한다.

인코딩된 입력 $\tilde{\mathbf{x}} \in \{0,1\}^{2n_{\text{in}}}$ 이 주어지면:

$$\begin{aligned}
\mathbf{a}^{(1)} &= \tilde{\mathbf{x}} \cdot C_{12}, & f_j^{(1)} &= \mathbf{1}[a_j^{(1)} > \theta_j^{(1)}] \\
\mathbf{a}^{(2)} &= \mathbf{f}^{(1)} \cdot C_{23}, & f_k^{(2)} &= \mathbf{1}[a_k^{(2)} > \theta_k^{(2)}] \\
\mathbf{s} &= \mathbf{f}^{(2)} \cdot C_{34}, & \hat{y} &= \arg\max_c\, s_c
\end{aligned}$$

2.3 카빙 규칙

$\hat{y} \neq y$ (잘못된 예측)일 때, 네트워크는 올바른 출력 클래스 y 방향으로 활성 경로를 깎는다:

Algorithm 1 협곡 카빙 (단일 샘플)

$h_2 \leftarrow \{k : f_k^{(2)} = 1\}$ 에서 임의 노드
$d_{h_2,y} \leftarrow \max(1,\ d_{h_2,y} - \alpha)$
$h_1 \leftarrow \{j : f_j^{(1)} = 1\}$ 에서 임의 노드
$h_2' \leftarrow H_2$ 에서 임의 노드 $\{h_2$ 와 독립적인 탐색 대상$\}$
$d_{h_1,h_2'} \leftarrow \max(1,\ d_{h_1,h_2'} - \alpha)$
$i \leftarrow$ 임의의 활성 입력 $(x_i = 1)$
$h_1' \leftarrow H_1$ 에서 임의 노드 $\{h_1$ 과 독립적인 탐색 대상$\}$
$d_{i,h_1'} \leftarrow \max(1,\ d_{i,h_1'} - \alpha)$

여기서 $\alpha > 0$ 은 카빙량(침식률)이다. 저항은 붕괴를 방지하기 위해 최소 1로 클램핑된다. 올바른 예측 시에는 카빙이 발생하지 않는다.

2.4 수면-회복

회복 메커니즘이 없으면 모든 저항은 결국 최솟값에 도달하여 네트워크를 포화시킨다. 각 단계 후, 평균 포화도 $1 - \bar{d}/d_0$ 가 역치(0.55)를 초과하면, 수면 단계가 모든 저항을 소량 β 만큼 균일하게 증가시키며, d_0 에서 클리핑한다:

$$d_{ij} \leftarrow \min(d_{ij} + \beta,\ d_0)$$

이는 수면 중 시냅스 통합을 모델링한다: 약하게 깎인 경로는 사라지고, 강하게 깎인 경로는 살아남는다.

2.5 역치 적응

10단계마다 발화 역치가 관찰된 활성화 범위의 중간점으로 갱신된다. 각 은닉층 $\ell \in \{1, 2\}$ 에 대해:

$$\theta_j^{(\ell)} \leftarrow \frac{\min_s a_{s,j}^{(\ell)} + \max_s a_{s,j}^{(\ell)}}{2}$$

이는 각 레이어의 노드가 균일하게 발화하지 않고 선택적으로 발화하도록 보장한다.

2.6 학습 절차

학습은 최대 E 에포크와 R 회의 랜덤 재시작으로 진행된다. 재시작 전체에서 가장 높은 점수의 네트워크를 유지한다. 하이퍼파라미터: $d_0 = 100$, $\alpha = 12$, $\beta = 3$, 포화 역치 $= 0.55$, $E = 1000$, $R = 50$.

3 실험

3.1 부울 게이트 학습

네 가지 기본 부울 함수에 대해 네트워크를 학습시킨다. 각 실험은 5개의 다른 랜덤 시드로 반복하며, 평균 정확도를 보고한다.

Table 1: 부울 게이트 학습 정확도 (5개 시드의 평균, $n_{h1} = n_{h2} = 8$, 에포크=1000, 재시작=50, $\alpha = 12$)

함수	입력	샘플	정확도
AND	2	4	1.000
OR	2	4	1.000
NOR	2	4	1.000
XOR	2	4	1.000

XOR이 핵심 사례다: 선형 분리가 불가능하고, 표현을 위해 은닉층이 필요하며, [6] 이후 비자명한 학습의 표준 벤치마크였다. River Learning은 단 한 번의 기울기 계산 없이 이를 푼다.

3.2 패리티 함수

n-비트 패리티는 까다로운 테스트다: 모든 비트가 필요하고, 어떤 입력도 무시할 수 없으며, 단일 레이어로 풀 수 있는 더 간단한 조각으로 분해할 수 없다.

Table 2: 패리티 학습 정확도†

과제	n	샘플	정확도
Parity-2	2	4	1.000
Parity-4	4	16	1.000
Parity-8	8	256	1.000

† Parity-n은 레이어당 $n_h = 16$ 은닉 노드를 사용하며, 이는 입력 공간의 조합적 증가(n-비트 패리티에 2^n 샘플)를 반영한다. 다른 모든 하이퍼파라미터(α, β, 포화 역치, 에포크, 재시작)는 표 1와 동일하다.

모든 패리티 함수가 이 설정에서 100% 정확도로 학습되며, 순방향 카빙이 다층, 비분해 함수를 처리할 수 있음을 확인한다.

4 논의

역방향 패스가 필요 없는 이유. 표준 역전파에서 기울기 $\partial\mathcal{L}/\partial w_{ij}$는 각 가중치에 정확히 어떻게 변해야 하는지를 알려준다. River Learning에서 카빙 규칙은 대신 잘못된 출력에 기여한 활성 경로를 식별하고 그 경로를 따라 저항을 줄여, 올바른 클래스에 대해 해당 경로를 강화한다. 신호는 국소적(어떤 경로가 발화했는가?)이고 이진(맞았는가 아닌가?)이지만, 여기서 테스트한 모든 함수를 학습하기에 충분하다.

물리적 해석과 Natural Optimum. 네트워크는 저항 그래프다. 올바르게 발화하는 경로에 전도도가 축적된다. 수면은 전체 포화를 방지한다. 이것이 정확히 물이 협곡을 깎는 방식이다: 흐름이 바위를 침식하고, 깎인 경로가 깊어지며, 다른 경로는 흐름을 덜 받아 채워진다. 전역 목적함수가 최소화되지 않는다; 최적 경로가 출현한다. 이것이 Natural Optimum이다 — 최적화 절차가 아니라 최적화 결과다. 시스템은 자신이 무엇을 최적화하고 있는지 모른다. 단지 흐를 뿐이다. 최적은 흐름 이후에 남는 것이다. 16개 노드로 초기화된 네트워크가 AND에서 3개로 수렴한다 — 어떤 시작점에서든 매번 동일한 최소 구조. 함수가 아키텍처를 결정했다. 하이퍼파라미터 탐색이 필요 없다.

추론과 학습의 분리 없음. 모든 기울기 기반 시스템에서 학습과 추론은 근본적으로 구별되는 연산이다: 학습은 저장된 계산 그래프와 역방향 패스를 필요로 하고, 추론은 동결된 네트워크에서의 읽기 전용 순방향 패스다. River Learning은 이 구분을 제거한다. 출력을 계산하는 동일한 순방향 패스가 활성 경로도 식별한다 — 그리고 뒤따르는 카빙 단계는 추론 시점에서 이미 존재하는 정보만 사용한다 (어떤 노드가 발화했는지, 출력이 올바른지). 추가 메모리도, 모드 전환도, 재계산도 필요 없다. 배포된 River 네트워크는 항상 학습 중이다. 이 속성은 엔지니어링 편의가 아니라 알고리즘의 구조적 결과이며, 지속적 학습, 온라인 적응, 그리고 학습과 추론 회로를 분리할 수 없는 물리적 기판 설계에 시사점을 가진다.

Forward-Forward와의 비교. Hinton의 Forward-Forward 알고리즘[3]도 역전파를 피하지만, 두 번의 순방향 패스(양성 및 음성 데이터)와 레이어별 적합도 목적함수가 필요하다. River Learning은 한 번의 순방향 패스와 단일 이진 출력 신호만 필요하다.

추측의 범위. River Learning은 문제에서 세 가지만 요구한다: (1) 정의된 입력 공간(입구), (2) 정의된 출력 공간(출구), (3) 주어진 경로가 올바른 결과를 생성했는지 평가하는 방법. 이 조건을 만족하는 모든 유한, 이산 입력 유향 네트워크 — 부울 회로, 라우팅 그래프, 시퀀스 변환기, 추론 체인 — 가 흐름 기반 학습의 후보다. 저항 매질과 카빙 규칙이 유일한 메커니즘이다. 문제를 풀기 위해 필요한 경로 구조는 설계되지 않고 출현한다. 형식적 지지는 두 단계로 접근할 수 있다: 첫째, $\{\mathrm{AND}, \mathrm{OR}, \mathrm{NOR}\}$이 기능적으로 완전한 기저임을 주목하면 모든 부울 함수가 이 네트워크에서 표현 가능하다; 둘째, 유한 저항 상태 공간에 대한 Lyapunov 논증이 유한 카빙 단계 내에 수렴을 시사한다. 엄밀한 증명은 향후 과제로 남긴다.

랜덤 재시작에 대하여. 랜덤 재시작($R = 50$)은 실용적 가속이지, 이론적 요구가 아니다. 16개 은닉 노드로 초기화된 네트워크가 AND에서 3개의 유효 노드로 수렴한다 — 초기화에 관계없이 어떤 시작점에서든 도달하는 동일한 최소 구조. 충분한 반복이 주어지면 단일 실행도 동일한 최적에 도달한다. 재시작은 흐름이 결국 스스로 완료할 과정을 가속할 뿐이다.

열린 문제. 형식적 수렴 한계와 풀 수 있는 문제 클래스의 정확한 경계는 열린 문제로 남아 있다.

5 결론

다층 네트워크가 부울 함수 — AND, OR, NOR, XOR을 100% 정확도로, n-비트 패리티를 $n = 8$까지 별도의 구성(표 2)으로 — 순방향 패스와 이진 정확성 신호만으로 학습할 수 있음을 보였다. 기울기도, 역방향 패스도, 보조 목적함수도 필요 없다. 이 방법은 물리적 비유(강의 협곡 침식)에 기반하며, 테스트한 모든 함수에서 완벽한 정확도를 달성한다.

이는 연구된 함수 클래스에 대해 순방향 경로 카빙이 역전파의 실행 가능한 대안임을 확립하며, 이 원리가 더 크고 연속 도메인 과제로 얼마나 확장될 수 있는지의 질문을 연다.

신경망 학습에서 빠져 있던 요소는 더 복잡한 기울기가 아니라 — 단순히 출구의 정의였다.

특허 및 협력

본 논문에 기술된 방법은 한국 특허 출원 **10-2026-0052624** (2026년 출원)에 의해 보호되며, 순방향 경로 카빙, 역치-반전 라우팅, 흐름 기반 경쟁적 억제에 대한 청구를 포함한다. PCT 국제 출원이 예정되어 있다.

저자는 다음을 환영한다:

- **학술 협력** — 본 연구를 확장하는 피어리뷰 저널 제출에 대한 공동 저자.
- **상업적 제안** — River Learning 또는 Natural Optimum에 기반한 라이센싱, 응용 연구, 또는 제품 개발.

연락처: 신연성 `whitepep@gmail.com`

References

[1] Francis Crick. The recent excitement about neural networks. *Nature*, 337(6203):129–132, 1989.

[2] Donald O Hebb. *The Organization of Behavior*. Wiley, 1949.

[3] Geoffrey Hinton. The forward-forward algorithm: Some preliminary investigations. *arXiv preprint arXiv:2212.13345*, 2022.

[4] Dong-Hyun Lee, Saizheng Zhang, Asja Fischer, and Yoshua Bengio. Difference target propagation. *Machine Learning and Knowledge Discovery in Databases*, pages 498–515, 2015.

[5] Timothy P Lillicrap, Adam Santoro, Luke Marris, Colin J Akerman, and Geoffrey Hinton. Backpropagation and the brain. *Nature Reviews Neuroscience*, 21(6):335–346, 2020.

[6] David E Rumelhart, Geoffrey E Hinton, and Ronald J Williams. Learning representations by back-propagating errors. *Nature*, 323(6088):533–536, 1986.

[7] Atsushi Tero, Seiji Takagi, Tetsu Saigusa, Kentaro Ito, Dan P Bebber, Mark D Fricker, Kenji Yumiki, Ryo Kobayashi, and Toshiyuki Nakagaki. Rules for biologically inspired adaptive network design. *Science*, 327(5964):439–442, 2010.

River Learning 시리즈

논문 1 역전파 없는 순방향 경로 카빙 (본 논문)
네트워크가 뒤를 돌아보지 않고 학습할 수 있는가?

논문 2 추론이 곧 학습: 위상 분리 없음 (*Zenodo, 2026*)
학습 모드와 추론 모드는 동일한 연산이다.

논문 3 하나의 게이트, 십만 개의 엣지: MNIST로의 확장 (*Zenodo, 2026*)
*101,632*개 엣지, 하나의 비선형 게이트, *90.4%* 정확도.

논문 4 수렴된 구조가 곧 설명이다 (*Zenodo, 2026*)
AND (*4→2*)에서 *MNIST* (*256→96*)까지: 살아남은 게이트가 설명이다.

논문 5 합류 게이트 네트워크 (*Zenodo, 2026*)
CGN: 생물학적 뉴런에서 표준 아키텍처로. 하나의 방정식, 아키텍처 하이퍼파라미터 제로.

논문 6 템플릿 공유와 학습으로부터의 네트워크 설계
같은 함수, 같은 구조. 한 번 배우고, 어디든 복사한다.
네트워크가 스스로를 설계한다.

한 편씩, 한 걸음씩.

부록: 참조 구현

아래 코드는 표 1를 재현하는 자체 완결적 Python 구현(NumPy만 사용)이다.

```
import numpy as np

def _cond(d, init_w=100.0):
    return np.minimum(1.0 / np.maximum(d, 0.01),
                      init_w / np.maximum(d, 1.0))

def dual_rail(data):
    result = []
    for x, y in data:
        xd = []
        for xi in x:
            xd += [xi, 1 - xi]
        result.append((xd, y))
    return result

class RiverNet:
    def __init__(self, n_in, n_h=8, n_out=2, init_w=100.0):
        self.n_in, self.n_h, self.n_out = n_in, n_h, n_out
        self.iw = init_w
        self._reset()

    def _reset(self):
        iw = self.iw
        r = lambda a, b: np.ones((a, b)) * iw + np.random.rand(a, b) * 20
        self.d12 = r(self.n_in, self.n_h)
        self.d23 = r(self.n_h,  self.n_h)
        self.d34 = r(self.n_h,  self.n_out)
        self.t1  = np.random.rand(self.n_h) * 0.5 + 0.1
        self.t2  = np.random.rand(self.n_h) * 0.5 + 0.1

    def _forward(self, X):
        C12 = _cond(self.d12, self.iw)
        C23 = _cond(self.d23, self.iw)
        C34 = _cond(self.d34, self.iw)
        a1 = X @ C12;               f1 = a1 > self.t1
        a2 = f1.astype(float) @ C23; f2 = a2 > self.t2
        return a1, f1, a2, f2, f2.astype(float) @ C34

    def score(self, data):
        X = np.array([x for x, _ in data], dtype=float)
        Y = np.array([y for _, y in data])
        *_, s = self._forward(X)
        return float(np.mean(np.argmax(s, 1) == Y))

    def _carve(self, data, amt=12.0):
        X = np.array([x for x, _ in data], dtype=float)
        Y = np.array([y for _, y in data])
        a1, f1, a2, f2, s = self._forward(X)
        for i in np.where(np.argmax(s, 1) != Y)[0]:
            y   = Y[i]
```

```
            ai  = np.where(X[i] > 0)[0]
            f1i = np.where(f1[i])[0]
            f2i = np.where(f2[i])[0]
            if len(f2i):
                self.d34[f2i[np.random.randint(len(f2i))], y] = max(1.,
                self.d34[f2i[np.random.randint(len(f2i))], y] - amt)
            if len(f1i):
                h1 = f1i[np.random.randint(len(f1i))]
                self.d23[h1, np.random.randint(self.n_h)] = max(1.,
                self.d23[h1, np.random.randint(self.n_h)] - amt)
            if len(ai):
                ii = np.random.choice(ai)
                self.d12[ii, np.random.randint(self.n_h)] = max(1.,
                self.d12[ii, np.random.randint(self.n_h)] - amt)

    def _tune(self, data):
        X = np.array([x for x, _ in data], dtype=float)
        a1, _, a2, _, _ = self._forward(X)
        for a, t in [(a1, self.t1), (a2, self.t2)]:
            lo, hi = a.min(0), a.max(0)
            m = hi > lo
            t[m] = (lo[m] + hi[m]) / 2

    def train(self, data, epochs=1000, restarts=50, erode=12.0):
        best, snap = 0.0, None
        for _ in range(restarts):
            self._reset()
            for ep in range(epochs):
                sat = 1 - np.mean(np.concatenate(
                    [d.ravel() for d in
                     [self.d12, self.d23, self.d34]])) / self.iw
                if sat > 0.55:
                    for d in [self.d12, self.d23, self.d34]:
                        np.minimum(d + 3., self.iw, out=d)
                self._carve(data, erode)
                if ep % 10 == 0:
                    self._tune(data)
            sc = self.score(data)
            if sc > best:
                best = sc
                snap = (self.d12.copy(), self.d23.copy(),
                        self.d34.copy(), self.t1.copy(), self.t2.copy())
            if best >= 1.0:
                break
        if snap:
            self.d12, self.d23, self.d34, self.t1, self.t2 = snap
        return best

# -- 표 1 재현 ------------------------------------------
AND = [([0, 0], 0), ([0, 1], 0), ([1, 0], 0), ([1, 1], 1)]
OR  = [([0, 0], 0), ([0, 1], 1), ([1, 0], 1), ([1, 1], 1)]
NOR = [([0, 0], 1), ([0, 1], 0), ([1, 0], 0), ([1, 1], 0)]
XOR = [([0, 0], 0), ([0, 1], 1), ([1, 0], 1), ([1, 1], 0)]
```

```
for label, data_raw in [('AND', AND), ('OR', OR),
                        ('NOR', NOR), ('XOR', XOR)]:
    data = dual_rail(data_raw)
    accs = []
    for seed in [42, 123, 7, 99, 0]:
        np.random.seed(seed)
        net = RiverNet(n_in=len(data[0][0]), n_h=8)
        accs.append(net.train(data))
    print(f'{label}  acc={np.mean(accs):.3f}')
```